DS605 · Fundamentals of Machine Learning

Lecture Notes 16-17

Regularization: Ridge, Lasso, and Elastic Net

Where we left off

Lecture 15: Fit ShopKart's delivery-time data with polynomials of increasing degree. The highest-degree curve tracked the training points almost perfectly — and that turned out to be the problem, not the solution. This lecture is one continuous argument in two parts:

- **Part 1.** Why flexible models overfit, and how *Ridge Regression* controls that by shrinking coefficients smoothly.
- **Part 2.** Why smooth shrinkage isn't always enough, and how *Lasso* and *Elastic Net* extend the idea — one of them all the way to dropping features entirely.

Part 1 — Why Regularization? Ridge Regression

1.1 The diagnostic question

> *"You're building a predictor. Its performance isn't good enough. What do you do?"*

ShopKart's delivery-time predictor is either too simple to capture the real pattern, or too eager to chase noise in the training data — either way, it isn't good enough yet. Two real fits on the same 26-point ShopKart sample make this concrete: a degree-1 line that's too simple, and a degree-15 curve that chases every point. Neither is what you want.

Your options, before naming any of them (and each has a catch):

1. Gather more training examples — *may not help.*
2. Remove noise in the training examples.
3. Adding more features or removing features *may, in some cases, worsen performance.*
4. Change model: move to a more complex or less complex model.
5. Keep the existing model but **add constraints** to reduce its complexity, or **remove constraints** to increase it.

Which of these helps depends entirely on which failure mode you're in.

If your model underfits: too simple to capture the pattern, even in data it has already seen, more training examples will *not* help. Your real options are to move to a more complex model, find or engineer additional predictive features, or remove constraints on the existing model to let it flex more.

If your model overfits: fitting the training data too closely, including its noise, your options flip: gather more data; clean up noisy examples; move to a *less* complex model; drop features; or add constraints to the existing model to hold it back.

Notice that "add constraints" and "remove constraints" show up on both lists, just in opposite directions. That one lever gets a name:

> Constraining a model to make it less complex and reduce the risk of overfitting is called **regularization**.

Regularization is a general idea, but the rest of this lecture develops it concretely for linear regression.

1.2 Two ways to reduce complexity

Linear models are among the *least* complex model families, which is why we usually associate them with underfitting rather than overfitting — but with enough features, some dimension that is actually irrelevant can look useful just by chance. If a linear model is overfitting, you need to reduce its **degrees of freedom**, and there are two ways to do it:

1. **Discard some features** entirely.
2. **Constrain the range of the coefficients** — let every feature stay in the model, but forbid any of them from growing too large.

The slide deck's live lab makes option 2 tangible: fit a flexible degree-12 curve to ShopKart's delivery-time data and drag a regularization slider (λ). At **λ near 0,** the curve chases every point, including noise (overfitting), and the coefficient bars are enormous. Push **λ up,** and the curve smooths out as the coefficients shrink toward zero. Push it **too far,** and the curve goes flat, missing the real pattern (underfitting). The same constraint causes both failure modes, just at opposite ends of one dial.

1.3 Writing the constraint into the loss function

Ordinary least squares minimizes squared error alone — how well the line fits the data, and nothing else:

$$J_{OLS}(\beta) = (1 / 2m) \cdot \Sigma_{i=1\ldots m} (\hat{y}_i - y_i)^2$$

Regularization adds a penalty on the **size** of the coefficients directly onto that loss — the first term still measures fit to the data; the second, scaled by λ, measures the size of β:

$$J_{reg}(\beta) = J_OLS(\beta) + \lambda \cdot \Omega(\beta)$$

In effect, this **penalizes hypotheses that fit the data too well** — the model must now earn every unit of coefficient magnitude, rather than simply chase the lowest possible training error. Everything from here is a choice of what Ω(β) actually is.

1.4 Ridge Regression: the L2-norm penalty

Ridge penalizes by the **L2-norm** of β, the sum of the squares of the coefficients:

$$\Sigma_{j=1 \ldots n} \beta_j^2$$

(Two notational notes: we never penalize β_0, the intercept, which is why the sum starts at j = 1; and the L2-norm itself is technically the square root of that sum, but what gets added to the loss is the sum of squares directly.)

Ridge Regression finds the β that minimizes:

$$J(X, y, \beta) = (1 / 2m) \cdot \Sigma_{i=1}^{m} (h_\beta(x^{(i)}) - y^{(i)})^2 + \lambda \cdot \Sigma_{j=1}^{n} \beta_j^2$$

λ is the **regularization parameter** — it sets how much weight the penalty carries relative to the fit term. Real Ridge fits at λ = 0, 10, 300 on ShopKart's data show the shrinkage directly: as λ climbs, the fitted curve relaxes from chasing every point toward a smoother, simpler shape.

1.5 Solving Ridge: the Normal Equation, with one added term

Ridge has a closed-form solution — the Normal Equation with one extra term, obtained the same way as ordinary OLS (take the gradient, set it to zero, solve for β):

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

Because β_0 isn't penalized, the "I" here isn't quite the identity matrix — it has a zero in the top-left corner. A useful bonus: whenever λ > 0, the matrix $(X^T X + \lambda I)$ is provably invertible — *even when* $X^T X$ alone is not. Ridge is therefore also a numerically safer way to solve regression problems with collinear or redundant features.

1.6 Solving Ridge: gradient descent

The update rule for every β_j except β_0 becomes:

$$\beta_j = \beta_j - \alpha \cdot [(1/m) \cdot \Sigma_{i=1}^{m} (h_\beta(x^{(i)}) - y^{(i)}) \times x_j^{(i)} + (\lambda/m) \cdot \beta_j]$$

which rearranges to:

$$\beta_j = \beta_j \cdot (1 - \alpha\lambda/m) - \alpha \cdot (1/m) \cdot \Sigma_{i=1}^{m} (h_\beta(x^{(i)}) - y^{(i)}) \times x_j^{(i)}$$

This rearrangement is the whole intuition in one line: **every gradient-descent step first multiplies β□ by a number just under 1, then applies the ordinary OLS correction.** That multiplicative shrink-first-then-correct pattern is exactly why Ridge coefficients get smaller as λ grows.

> **Convention note.** This update rule (and the Normal Equation above) comes out cleanest when λ is folded into the same 1/2m as the fit term — i.e. (1/2m)·[Σ(·)² + λΣβ□²] — rather than added on afterward. Different textbooks place λ on either side of that bracket; it only rescales what a given numeric value of λ means, not how Ridge behaves.

1.7 Implementing Ridge in Python

```python
from sklearn.linear_model import Ridge

ridge_reg = Ridge(alpha=1, solver="cholesky")
ridge_reg.fit(X, y)
ridge_reg.predict([[1.5]])
# array([[1.55071465]])
```

Or, equivalently, with stochastic gradient descent:

```python
sgd_reg = SGDRegressor(penalty="l2")
sgd_reg.fit(X, y.ravel())
sgd_reg.predict([[1.5]])
# array([1.47012588])
```

`solver` can be set explicitly (Cholesky, SVD, etc.) or left at scikit-learn's default, `"auto"`. As with most regularized methods, **scaling your feature values first is strongly advised** — the penalty term is scale-sensitive, so unscaled features get penalized unevenly.

1.8 Part 1, in one breath

Ridge addresses overfitting by discouraging unnecessarily large coefficients, smoothly and with a clean, closed-form solution. But look again at the shrinkage factor from gradient descent — β□ · (1 − αλ/m) — and notice what it *never* does: no matter how large λ gets, it keeps multiplying β□ by a number strictly between 0 and 1. **A Ridge coefficient can get arbitrarily close to zero, but never quite lands on it.**

That observation is the hinge into Part 2: what if the *penalty itself* — not just λ — changed, so that some coefficients could be driven to exactly zero?

Part 2 — Lasso, Sparsity, and Elastic Net

Same ShopKart data, same fit-error term, same gradient-descent machinery as Part 1. The only change is the penalty — from a sum of squares to a sum of absolute values — and that single change produces a completely different, non-smooth kind of behavior: coefficients that land on exactly zero.

2.1 Lasso Regression: the L1-norm penalty

Lasso stands for *least absolute shrinkage and selection operator* (the name itself doesn't matter much). It penalizes by the **L1-norm** of β — the sum of absolute values of the coefficients:

$$\Sigma_{j=1}^{n} |\beta_j|$$

(again, β_0 is excluded, hence j starting at 1). Lasso Regression finds the β that minimizes:

$$J(X, y, \beta) = (1 / 2m) \cdot \Sigma_{i=1}^{m} (h_\beta(x^{(i)}) - y^{(i)})^2 + \lambda \cdot \Sigma_{j=1}^{n} |\beta_j|$$

λ plays the same balancing role it did for Ridge — how much the loss trades off fit against penalty — just multiplying a different function of β.

2.2 The regularization parameter, revisited

Whether it's called λ (the textbook convention) or `alpha` (scikit-learn's argument name for it), this hyperparameter controls the balance between fitting the data and parameter shrinkage. Choosing it carelessly costs you either fit quality (too large) or overfitting protection (too small) — model selection for λ is itself a recurring theme once both Ridge and Lasso are on the table.

2.3 As λ grows, some β are driven to exactly zero

This is Lasso's defining behavior: the fitted model treats some features as **irrelevant** — Lasso performs a form of automatic **feature selection** that Ridge never does. Real Lasso fits at λ = 0, 0.05, 2 on the same ShopKart data (degree-12 polynomial features, standardized) show this directly: as λ increases, individual coefficient bars don't just shrink — they snap to exactly zero, one at a time, while the survivors keep doing the work.

2.4 Implementing Lasso in Python

```python
from sklearn.linear_model import Lasso

lasso_reg = Lasso(alpha=0.1)
lasso_reg.fit(X, y)
lasso_reg.predict([[1.5]])
# array([1.53788174])

# Equivalently:
# SGDRegressor(penalty="l1")
```

As before, scale your features first. For polynomial regression specifically, the usual pipeline is: expand with `PolynomialFeatures(degree=...)`, scale with `StandardScaler`, *then* apply the regularized regressor.

2.5 Ridge vs. Lasso: not the same kind of shrinkage

Both methods shrink coefficients, but differently:

- **Ridge** shrinks **proportionally** — every coefficient is scaled down by roughly the same factor, and none of them (in general) reach exactly zero.
- **Lasso** shrinks **by a constant amount**, which is precisely what lets small coefficients get knocked all the way to zero while larger ones survive, reduced.

On the same 12 real fitted coefficients from ShopKart's data, Ridge ($\lambda = 30$) shrinks everything smoothly, while Lasso ($\lambda = 0.3$) zeroes out several of them outright.

A side note worth remembering: unregularized OLS at this same degree-12 fit is wildly unstable — coefficients as large as 3.8×10^9 in magnitude, because 26 data points can't pin down 12 correlated polynomial features. **That instability is itself an argument for regularizing**, independent of the bias/variance tradeoff — it's also why an unregularized comparison can't share an axis with the well-behaved Ridge/Lasso bars.

Fitting an unregularized model and both regularized models on the same 26 ShopKart points side by side makes the contrast visual: OLS chases noise, Lasso and Ridge both look far more reasonable, and Lasso's curve corresponds to a model actually using fewer effective features.

2.6 Why this happens: the graphical interpretation

Minimizing $\text{Loss}(\beta) + \lambda \cdot \text{Complexity}(\beta)$ is mathematically equivalent to minimizing $\text{Loss}(\beta)$ **subject to the constraint** $\text{Complexity}(\beta) \leq c$, for some constant c tied to λ. Geometrically:

- **L1 (Lasso)** constrains β to a **diamond** — the region $|\beta_1| + |\beta_2| \le c$.
- **L2 (Ridge)** constrains β to a **circle** — the region $\beta_1^2 + \beta_2^2 \le c^2$.

The optimum is where the (tilted, elliptical) contours of the loss function first touch the boundary of the allowed region as it shrinks.

> **The key geometric fact.** The diamond has **corners** sitting exactly on the axes. As the constraint region shrinks, a tilted error ellipse will generically touch a **corner** first, and a corner means one coordinate is exactly zero. The circle has no corners: the ellipse can touch it anywhere along a smooth curve, which essentially never lands exactly on an axis. **That difference alone is the entire geometric reason Lasso is sparse, and Ridge isn't.**

2.7 Generalizing further: Lq balls

Ridge's and Lasso's penalties are two members of a continuous family, indexed by an exponent q:

$$|\beta_1|^q + |\beta_2|^q \le c^q$$

- q = 2 gives the circle — Ridge.
- q = 1 gives the diamond — Lasso, and notably **the smallest value of q for which the constraint region is still convex**, which is exactly why Lasso's optimization problem stays well-behaved.
- Below q = 1, the region caves in around the axes and is no longer convex.
- q = 4 (or any q > 2) rounds the shape outward into something closer to a square with rounded corners.

Dragging q across this range shows Ridge and Lasso not as two unrelated techniques but as two points on a single continuous dial.

2.8 Elastic Net: combining both

Elastic Net mixes the L1 and L2 penalties with a mixing hyperparameter, on top of the existing λ:

$$\lambda \cdot \Sigma_{□=1 \ldots n} \left(\alpha \cdot \beta_□^2 + (1 - \alpha) \cdot |\beta_□| \right)$$

Its constraint region sits geometrically **between** the diamond and the circle — a rounded diamond, at whichever q the mixing weight α effectively corresponds to on the Lq dial above.

```python
from sklearn.linear_model import ElasticNet

elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)
elastic_net.fit(X, y)
elastic_net.predict([[1.5]])
# array([1.54333232])
```

2.9 Closing the loop

That's regularization for linear regression — Ridge, Lasso, and Elastic Net, all built on exactly the same OLS loss with one added penalty term, differing only in the shape of Ω(β).

> **Next lecture:** Logistic Regression — moving from predicting continuous values to predicting probabilities of a binary outcome.

Quick-reference summary

	Ridge	**Lasso**	**Elastic Net**
Penalty Ω(β)	Σ□ β□² (L2)	Σ□ |β□| (L1)	Σ□ (α·β□² + (1−α)·|β□|)
Constraint region shape	Circle (no corners)	Diamond (corners on axes)	Rounded diamond, between the two
Drives coefficients to exactly zero?	No — shrinks proportionally, asymptotes toward zero	Yes — shrinks by a constant amount, snapping small coefficients to zero	Yes, tunable via α
Performs feature selection?	No	Yes	Partially, tunable
Closed-form solution?	Yes — modified Normal Equation	No — the L1 term is non-differentiable at 0	No
scikit-learn class	`Ridge` (or `SGDRegressor(penalty="l2")`)	`Lasso` (or `SGDRegressor(penalty="l1")`)	`ElasticNet`
Good default when…	Many small/moderate effects; multicollinearity	You suspect only a few features actually matter	You want some sparsity, but Lasso feels unstable / features are correlated

Key terms

- **Regularization** — constraining a model to reduce its complexity, in order to reduce the risk of overfitting.
- **Regularization parameter (λ / `alpha`)** — the weight on the penalty term relative to the fit term.
- **L1-norm / L2-norm** — sum of absolute values/sum of squares of the coefficients, respectively (up to the root, in the strict norm definition).
- **Sparsity** — a fitted model in which some coefficients are exactly zero, i.e., some features are effectively dropped.
- **Degrees of freedom** — informally, how many independent ways a model is free to vary to fit data; regularization and feature removal both reduce it.